Introduction to Bash Scripting

Introduction to the Shell and Bash

What is a shell?

A shell is a program that provides a command-line interface allowing users to interact with their operating system. It interprets the commands you type and translates them into actions the computer can perform. Essentially, you are communicating with you computer in a language you and your computer both understand. There are many types of shells available, each wth its own syntax and features. Some popular ones include sh (Bourne shell), csh (C shell), ksh (Korn shell), and bash (Bourne Again SHell).

What is Bash?

As stated above, Bash stands for Bourne Again SHell. It is one of the most commonly used Unix shells today, especially on Linux and macOS systems.

Bash is a command language interpreter that allows you to:

- Run commands interactively at a prompt.
- Write scripts that automate sequences of commands.
- Manage variables, control flow, and input/output redirection in scripts.

It combines features from earlier shells, offering a powerful and flexible environment for seamlessly moving around your local machine and HPC systems as well as scripting.

In another life, I would have been a history professor. As such, we will delve into a bit of the history surrounding the origin of bash scripting.

Shell Scripting Historical Background

The Bourne Shell (sh) was developed by Stephen Bourne at AT&T Bell Labs in 1977. The Bourne shell became the standard Unix shell for many years because of its scripting capabilities, allowing user to automate tasks in Unix environments.

In the 1980s, other shells emerged with new features and improvements. The C shell (csh), for example, provided a syntax resembling the C programming language, and the Korn shell (ksh) introduced advanced scripting features that appealed to developers.

By the late 1980s, with the rise of the GNU Project, there was a need for a free, open-source shell that combined the power of Bourne shell with enhancements fom Korn and C shells. This led to the invention of bash by Brian Fox in 1989.

Bash was designed as a replacement for the Bourne, compatible with its syntax but incorporating many modern features such as command line editing, improvided scripting syntax, and better user interactivity. It became the default shell on most Linux machines and later on macOS.

Why should we be using bash scripting in data science?

As budding data scientists, much of our work relvolves around managing files, running analyses, and automating repetitive tasks across different computing environments. Bash scripting is a powerful tool to help us do all this efficiently. Bash scripting is particualrly useful for a number of tasks you will encounter in your workplaces:

Efficient File and Directory Management

Using bash scripting we can quickly navigate directories and organize files. We can also rename, move, or delete files in bulk. One other thing I have found particularly use file is extracting or combining parts of files using tools like grep, awk, and cut.

Automation and Reproducibility

Bash scripts allow us to automate sequences of commands, reducing manual effort. If your scripts are well commented and automated, you can pass them along to your friends and collegues to use on their machines as well. This enhances reproducibility, which is what we strive for in science.

Interfacing with High Performance Computing

In your positions you might find yourself working on internal HPC systems. HPC systems usually require jobs to be submitted via command-lin job schedulers such as SLURM (Dartmouth). Bash scripts provide a way to write job submission scripts that define job resources, load software modules, and run your code or pipeline. This is a great way to prevent human error and not use all the RAM on your local machine to process your data!

Integration with Other Tools and Languages

We can use bash scripting to call to Python, R, or other data analysis tools. We can also use command line utilities for data processing (sed, awk, sort, etc.)

Basic Commands

Before we jump into bash commands, you need to access a terminal (command-line) environment where you can type and run these commands.

Mac Folks

Open Finder. Navigate to Applications -> Utilities. Double click on Terminal to launch it.

If you can't find the Terminal that way, you can use Spotlight Search (command + space), type Terminal, and press Enter. A terminal window will open, showing a prompt where you can enter commands.

Windows Folks

Windows does not natively include a Unix-style shell, but you can use Windows Subsystem for Linux (WSL). WSL allows us to run a genuine Linux environment, including bash and most command-line tools, directly on Windows 10 or 11. This menas that Windows users can run bash scripts, use Unix commands, and nstall Linus software alongside their Windows apps.

The Microsoft documentation you need to install this can be found (here)["https://learn.microsoft.com/en-us/windows/wsl/about"]. This link provides step-by-step installation instructions and guidance on using your favorite Linux distributions (Ubuntum Debian, Kali, openSUSE, etc.) on Windows.

Basic Commands and How to Use Them

There are a few sets of commands I would like to cover in today's lecture: navigating filesystems, viewing file contents, and file and directory manipulation. We will begin with navigating filesystems.

Navigating Filesystems

1) pwd Once you have your terminal open on your computer, go ahead and enter this command:

```
pwd
```

What does this return? It should be something like:

```
/Users/shira/
```

The `pwd` command returns our current directory. This is the location that we are at in our computer. The equivalent of this would be us opening up our finder and selecting our Desktop. This means that our computer can see all of the files we have on our Desktop.

Let's say we want to store our current directory as variable so that we can access it easily:

```
current_dir=$(pwd)
echo "You are currently in: $current_dir"
```

In the above directory we have stored the path of our current directory in a variable called "current_dir". The `$` is used to reference the value of a variable. When we set the `current_dir` we are referencing the output of `pwd`. When we are using the value stored in the `current_dir` later, we use the `$` to access the stored value.

2) ls

`pwd` gave us the power to see where we are in our machine. `ls` gives us the power to list the files in our current directory.

Go ahead and type in the following command:

```
ls
```

This command should return a list of all the files in your current directory. For me, it lists all of the files on my Desktop.

Now, computers do some funky things, including creating hidden files that we typically do not see in our Finder. If we want to see what hidden files are in our current directory we can type the following command:

```
ls -lah
```

`-lah` is a series of flags that mean the following:

`-a` - shows hidden files (`.` prefix) `-l` - uses long listing format `-h` - formats sizes human-readably

3) cd

We can't stay in our current directory forever! What if we want to take a look at the file in our Documents directory or perhaps in a particular project directory, we can do this with the `cd` command.

The following command can be used to get to my home directory from anywhere:

```
cd ~
```

We can also move up one directory with the following command:

```
cd ..
```

We can also use absolute paths in order to jump from one location using `cd` to another. You can simply insert the path to which you would like to navigate like this:

```
cd /Users/shira/Desktop/
```

This works with both relative and absolute path names.

```
cd Desktop
```

`cd ..` goes up a level.

4) mkdir

`mkdir` creates a new directory.

```
mkdir my_new_project
```

We can leverage flags once more to create nested directories in one command:

```
mkdir -p my_new_project/scripts/preprocessing
```

5) cp

The `cp` command is used to copy files from one directory to another. To test out this command, let's go into a text editor and make a text file containing a bit of our own data. I've added a .txt file to my Desktop called "test.txt". In this file I've just added a sentence. Now I can use the `cp` command to copy this file from my Desktop to my Documents I can implement the following command:

```
cp /Users/shira/Desktop/test.txt /Users/shira/Documents/test2.txt
```

We can also copy entire directories from location to another location using the `cp` command along with the `-r` flag which stand for "recursive". To do this we run the following command:

```
cp -r source_folder destination_folder
```

You can se the source directory here to, perhaps, the directory for this class that you have on your local machine and you can copy it over to your GitHub directory or some other location.

6) mv

`mv` moves or renames files or directories.

If this command is implemented on a file in the same directory it will rename this file. Run this line on that file we previously created (MAKE SURE YOU ARE IN THE CORRECT DIRECTORY!):

```
mv test.txt test_new_name.txt
```

7) cat

This command prints the contents of a file in our current directory. It can also print the contents of a file in a different directory if we provide the command an absolutely path to that file:

```
cat test_new_name.txt
```

This should print out the contents of our file.

8) rm

The last command we will discuss in this section is the `rm` command. This command deletes files or directories. To delete the test file we created, we can run this command:

```
rm test_new_name.txt
```

If we were hoping to delete an entire directory we can do so using the `-r` flag. This flag removes everything in the directory recursively.

```
rm -r folder_to_delete
```

Here is a summary of the commands we used in this section:

Command	Purpose	Brief Description
pwd	Print Working Directory	Shows your current directory path
ls	List files and directories	Lists contents of a directory
cd	Change Directory	Moves through the directory tree
mkdir	Make Directory	Creates new directory/folder
cp	Copy files or directories	Copies files or folders
mv	Move or rename files or directories	Moves or renames files/folders
rm	Remove files or directories	Deletes files or folders permanently
cat	Display file contents	Prints text file content to terminal

Let's navigate to where your data files are so we can work with them directly. For me, it's:

```
cd HDS711/Week_4/data
```

- Viewing File Contents

In the previous section we used the `cat` command to display everything in our file, but what if we just wan to display part of that file? Afterall, some of the files we will be working with will be exceptionally large and we don't want to pint out everything in the file. In order to do this we can use the following commands:

1) head

The `head` command shows the first few lines of a file, By default, it displays the first 10 lines. I have provided you with a sample RNA-seq count matrix in the data directory for this week on my GitHub. This file is also available on our Canvas page. Download this file and save it somewhere on your local machine where you can find it. I'll be referenceing this file in my GitHub directory on my local machine. Let's use the `head` command to display the first few lines of this file.

```
head data.csv
```

The return from this command should display the column names, which are sample names, and the rows, which are gene names. You should also see some pseudo count data for each gene and sample pair.

What if you want to display more than just the first 10 lines of the file? We can use the flag `-n` to indicate the number of lines we would like to print out. For example, the code below displays the first 20 lines of the file.

```
head -n 20 data.csv
```

2) tail

Antithetical to `head`, `tail` displays the last few lines of a file and also defaults to 10 lines.

```
tail data.csv
```

The above code will display the last 10 lines of our file.

```
tail -n 20 data.csv
```

As we did with the `head` command we can ask for more than 10 lines to be displayed in the code above.

3) less

The less command lets you open a file and navigate it one screen at a time.

```
less data.csv
```

When you run this command, you will observe the the contents of the file fill your terminal. While the file is showing, you can use the Page Up/Down arrows to scroll through. You can also use `/search_term` to find occurences of certain words or phrases.

```
/"Gene123"
```

To exit, simply press `q` on your keyboard for "quit".

4) wc

Though not strictly a viewing command, `wc` provides quick stats about file size, word count, and line count which help when insecting a data set.

```
wc data.csv
```

The above code displays the lines, words, and bytes in our file. If we just want the lines we can use the `-l` flag.

```
wc -l data.csv
```

This line of code displays just the number of lines in our data file.

Command	Purpose	Brief Description
head	Show first lines of a file	Preview the start of a file (default 10 lines)
tail	Show last lines of a file	View end of a file; can monitor live updates (`-f`)
less	Interactive file viewer	Scroll, search, and navigate large files
wc	Count lines, words, bytes in a file	Quickly get file size stats

Bash Scripting Basics

Bash scripting allows you to automate sequences of commands and build programmable workflows. This section introduces fundamental scripting concepts, including file and directory maniulation and common shell operations like piping and output redirection.

Writing and Running a Bash Script

In a video associated with this class, we learned how to submit a job on the HPC system using a bash slurm submission script. The script we had this job run was an R script. We can also create files to run bash scripts themselves.

Let's create a bash script that makes a directory for an imaginary project. Within that directory, we will create a subdirectory called "data". We will also create a new file that contains our project notes. We will then copy the data.csv file we were previously working with into our newly created "data" subdirectory and then rename that file.

Copy and paste the below script into a new file and save that file as "my_first_bash_script.sh". Make sure that you correct the paths to be specific to your machine!

The symbols #! together is called a "shebang" and it tells computer what language to read script as.

```
#!/bin/bash # This line tells your computer to recognize the bash scripting language

# Create a project folder
mkdir -p my_project/data

# Create an empty text file to store notes
touch my_project/notes.txt

# Copy a file into the new data directory (assuming 'data.csv' exists)
cp HDS711/Week_4/data/data.csv my_project/data

# Move the notes file to the data directory (renaming it at the same time)
```

```
mv my_project/data/data.csv my_project/data/rnaseq_dat.csv

# List the contents of the data folder
ls -l my_project/data/
```

In order to make this script executable you must run the following line:

```
chmod +x ./my_first_bash_script.sh
```

Then you can run the script like this:

```
./my_first_bash_script.sh
```

Make sure you are in the directory in which your file is located in order to run it! Otherwise, you must refer to the absolute path where thw file is located.

- Piping and Output Redirection

One command we have yet to discuss if the `grep` command. The `grep` command searches for words or phrases in a string.Piping (`|`) sends the output of one command as input to another. Let's combine these two commands to list all of the files in our directory and then find the files that contain only the extension ".csv"

```
ls -l HDS711/Week_4/data/ | grep ".csv"
```

The only file that I have in the directory that I reference in the previous code is "data.csv". If I simply input everything before the `|` the code will return "data.csv". If I only ran everything after the `|`, I will also return the "data.csv" file. The pipe command take the output from the listing of the files and inputs it to the grep command. If I had had `grep ".txt"`, the code would return nothing.

We can also write out the output of commands, even piped commands, to new files.

In the below code, we write out the output from the command we just ran above to a file on our Desktop.

```
ls -l HDS711/Week_4/data/ | grep ".csv" > files_list.txt
```

Two other commands I would like to introduce here are the `awk` and `cut` commands. The `awk` command extract or transform columns and perform calculations. The `cut` command extracts specific columns based on a delimeter.

This example prints the second column of a .csv file:

```
awk -F',' '{print $2}' HDS711/Week_4/data/data.csv
```

Similarly, this code extracts columns 1 and 3 from our .csv file.

```
cut -d',' -f1,3 HDS711/Week_4/data/data.csv
```

Control Structures and Applications to Data Science

Control structures allow your bash scripts to make decisions and perform repetitive tasks - key for automating data science workflows.

- Conditional Statements: if

The `if` statement lets our script execute commands based on conditions, much like in R programming. The general anatomy of an `if` statement in bash is outlined below. `fi` is if backwards, and it indicates teh end of the if/then statement.

```
if [ condition ]; then
  # commands if true
else
  # commands if false (optional)
fi
```

Let's create a simple example in which we search for our data.csv file in the directory in which it is stored:

```
if [ -f "HDS711/Week_4/data/data.csv" ]; then
  echo "data.csv exists."
else
  echo "data.csv not found."
fi
```

Loops

- for loops are useful for iterating over a list of items.

In the following example, we loop through each file with a ".csv" extension and execute command on each of the files.

```
#cd HDS711/Week_4/data/ # If you are not in the right location, get there

for file in *.csv; do
  echo "Processing $file"
  head -n 5 $file
done
```

- while loops repeat commands while a condition is still true. Take a look at the example below:

```
count=5
while [ $count -gt 0 ]; do
  echo "Countdown: $count"
  count=$((count - 1))
done
echo "Blast off!"
```

This script counts down from 5 and when that condition is no longer met, the loop exits and then prints out "Blast off!". Note `-gt` is "greater than".

Using Bash Scripting to Run R Code

```
#!/bin/bash
Rscript my_script.R
```

Or with arguments:

```
#!/bin/bash
Rscript my_script.R input.csv output.csv 10
```

And then in R you can capture the arguments and use them in your script:

```
args <- commandArgs(trailingOnly = TRUE)
input_file  <- args[1]
output_file <- args[2]
n           <- as.numeric(args[3])
```

and you can save the output to a file in R like we've being doing `write.csv(df, output_file)`

or direct script output (whatever it prints) to a file: `Rscript my_script.R > output.txt`

`>` overwrites the file each time. Use `>>` to append.

You can also capture the printed output in a bash variable:

```
  result=$(Rscript my_script.R)
```

You can also start your script with `#!/usr/bin/env Rscript` and it will know to use R to run it.

Summary

This concludes our foray into bash scripting. Here is a summary of everything we covered in this lecture!

1. Introduction to the Shell and Bash

- A shell is a command-line interface allowing users to interact with their computer's operating system by interpreting typed commands.

2. Why Use Bash Scripting in Data Science?

- File and Directory Management: Quickly navigate, organize, rename, move, and delete files/folders, or extract data parts using tools like grep, awk, and cut.
- Automation and Reproducibility: Automate repetitive tasks to save time and share workflows, ensuring reproducible analyses.
- High Performance Computing (HPC): Submit jobs and manage analysis workflows efficiently on HPC systems via command-line schedulers (e.g., SLURM).
- Integration: Bash scripts can call other tools and languages, facilitating flexible, powerful pipelines.

3. Basic Commands and Their Usage

- Common commands to navigate and manipulate the filesystem: pwd — print current directory ls — list directory contents (including hidden files with ls -lah) cd — change directory; supports relative and absolute paths (cd ~, cd ..) mkdir — create directories (mkdir -p for nested) cp — copy files or directories (-r for recursive) mv — move or rename files/directories rm — remove files or directories (-r for recursive) cat — display file contents
- Display and explore file contents efficiently: head — show first lines (head -n for custom line count) tail — show last lines, with option to monitor file live (tail -f) less — interactive file viewer with scrolling and search wc — count lines, words, bytes

4. Bash Scripting Basics

- Writing bash scripts with shebang #!/bin/bash.
- Using variables (assignment without $, referencing with $).
- Example script: create directories, make empty files (touch), copy and rename files.
- Using piping (|) to send output of one command to another (e.g., ls | grep ".csv").
- Using output redirection (>) to save files.

5. Control Structures

- Conditional statements (if) allow executing code blocks based on tests (e.g., check file existence).
- Loops:
- for loops to iterate over file lists (e.g., all .csv files).
- while loops to repeat commands while conditions are true (e.g., countdown example).

6. How to Run R Scripts Using Bash